

Pedagogical Report

Image Captioning with Vision-Language Models

INFO 7390: Advanced Data Science and Architecture

Northeastern University

Prepared by: Abhinav Kumar Piyush

December 2025

1. Teaching Philosophy

1.1 Target Audience

This workshop is designed for graduate students in data science, machine learning, or related technical fields who are seeking to expand their expertise into multimodal AI systems. The target audience possesses intermediate programming skills and foundational knowledge of deep learning concepts, but may not have prior experience with vision-language models or the specific challenges of bridging visual and textual modalities.

Understanding the target audience is crucial for effective instructional design. Pitching content too low results in disengagement; pitching too high creates frustration. This workshop assumes a specific skill baseline while remaining accessible to learners at the upper boundary of that baseline.

Background Assumptions

The specific prerequisite knowledge and skills expected from learners include:

- **Python Proficiency (Intermediate Level):** Students should be comfortable with functions, classes, type hints, list comprehensions, decorators, and object-oriented programming patterns. They should be able to read and write Python code without significant assistance, debug common errors independently, and understand Pythonic idioms. Experience with virtual environments and package management (pip, conda) is expected.
- **Deep Learning Fundamentals:** Understanding of neural network basics including layers, activation functions (ReLU, softmax), loss functions (cross-entropy), backpropagation, and gradient descent optimization. Students should grasp the distinction between training and inference, understand overfitting/underfitting, and be familiar with batch normalization and dropout concepts.
- **PyTorch Experience:** Basic experience with PyTorch tensors, model loading, and running inference. Students should understand device management (CPU/GPU), the purpose of `torch.no_grad()` for inference, and how to work with pre-trained models. Prior experience with training loops is helpful but not required since this workshop focuses on inference.
- **NLP Basics:** Foundational knowledge of tokenization (converting text to numerical representations), word embeddings (dense vector representations), and the general concept of attention mechanisms. Prior exposure to transformer architectures (BERT, GPT) provides useful context but is not strictly required.
- **Mathematical Maturity:** Comfort with linear algebra concepts including matrices, vectors, dot products, and matrix multiplication. Understanding of basic probability (distributions, sampling) and the softmax function. Students should be able to follow mathematical notation without requiring line-by-line derivations.
- **Development Environment:** Familiarity with Jupyter notebooks or similar interactive environments. Ability to navigate file systems, install packages, and troubleshoot common environment issues. Access to either local Python installation or cloud resources (Google Colab).

1.2 Learning Objectives

Upon completion, students will demonstrate the following competencies organized by Bloom's Taxonomy:

Bloom's Level	Learning Objective	Demonstration
Understanding	Explain encoder-decoder architecture and cross-attention mechanism	Interpret diagrams; answer conceptual questions
Applying	Implement image captioning pipeline using pre-trained BLIP	Write functional code that generates captions

Analyzing	Analyze how generation parameters affect output quality	Conduct experiments; interpret results
Evaluating	Evaluate caption quality using BLEU and semantic similarity	Compare metrics; identify disagreement cases
Applying	Apply GIGO principles to detect input quality issues	Build quality analyzer with appropriate checks
Creating	Design production-ready pipelines with error handling	Implement complete pipeline class

1.3 Pedagogical Approach and Rationale

This workshop employs a constructivist pedagogical framework, grounded in the understanding that meaningful learning occurs when students actively construct knowledge through engagement with material rather than passively receiving information. The instructional design draws on established learning theories including Vygotsky's Zone of Proximal Development, Bloom's Taxonomy, and principles of multimedia learning. The design follows three core principles:

The Explain-Show-Try Framework

Every major concept follows a deliberate three-stage progression designed to scaffold understanding:

- **Explain (Conceptual Foundation):** Theoretical foundations are presented with multiple representations including visual diagrams, analogies to familiar concepts, and clear explanations emphasizing 'why' not just 'what'. For example, cross-attention is explained through the analogy of describing a photo to a friend — your eyes naturally focus on different regions as you mention different elements. This grounds abstract mathematical concepts in intuitive experience.
- **Show (Worked Examples):** Live code demonstrations with comprehensive comments show concepts in action. Each code block is designed to be simultaneously copy-paste ready and educational. Variable names are descriptive, functions include detailed docstrings, and comments explain the purpose of each section. Students observe correct implementation patterns before attempting their own.
- **Try (Guided Practice):** Scaffolded exercises progress from fill-in-the-blank style (testing recall) to open-ended implementation challenges (testing synthesis). Early exercises provide substantial structure; later exercises require design decisions. Solutions are provided separately to enable self-assessment.

Progressive Complexity

Following Vygotsky's Zone of Proximal Development, complexity builds gradually. We begin with a simple three-line caption generation that produces immediate, satisfying results — this early success builds confidence and motivation. We then progressively layer on: conditional prompting, parameter tuning, evaluation metrics, quality analysis, confidence estimation, and finally production pipeline design. Each layer builds on previous understanding, preventing cognitive overload while maintaining engagement through appropriately increasing challenge. Students are never asked to take leaps beyond their current capability.

Immediate Visual Feedback

Unlike many ML tasks where results are abstract numbers requiring interpretation, image captioning provides immediate, intuitive feedback. Students see an image and generated caption — they can instantly judge whether the model 'understood' correctly. This tight feedback loop accelerates learning by making connections between code changes and outcomes immediately visible. The Gradio interactive demo enhances this by allowing real-time parameter experimentation, building intuition faster than batch experiments.

Multiple Entry Points

Recognizing that students enter with varying backgrounds and learn at different paces, the workshop provides multiple pathways. The main tutorial follows linear progression, but students can skip ahead to exercises or spend extra time with explanations. The Gradio demo provides an alternative entry point for kinesthetic learners who prefer exploration over instruction. Reference solutions allow struggling students to study correct implementations.

2. Concept Deep Dive

2.1 Technical and Mathematical Foundations

Image captioning requires bridging two fundamentally different data modalities: continuous visual signals (pixel values in high-dimensional space) and discrete linguistic tokens (words from a finite vocabulary). This section details the technical foundations enabling this challenging bridging task.

The Encoder-Decoder Paradigm

The system employs an encoder-decoder architecture, a design pattern with roots in information theory and neural machine translation. The encoder processes the input image into a dense, information-rich intermediate representation — a compressed encoding capturing semantically meaningful features while discarding irrelevant details. The decoder converts this representation into natural language, 'reading' the encoding and 'translating' it into words. This separation allows specialization: the encoder becomes expert at visual feature extraction, the decoder at conditioned language generation.

Vision Transformer (ViT) Architecture

The image encoder uses Vision Transformer architecture, a breakthrough demonstrating transformer models could achieve state-of-the-art image recognition. The key innovation is treating an image as a sequence of patches rather than a pixel grid. A 384×384 pixel image is divided into 576 non-overlapping 16×16 patches (24×24 grid). Each patch is flattened and linearly projected to a 768-dimensional embedding vector. Positional encodings preserve spatial information — without these, the model couldn't distinguish patches from different locations. A special [CLS] token aggregates global image information. The 577-token sequence passes through 12 transformer layers with self-attention, building increasingly abstract representations from low-level features (edges, textures) to high-level concepts (objects, relationships).

Cross-Attention: Bridging Vision and Language

The critical mechanism connecting modalities is cross-attention. While self-attention allows tokens to attend within the same sequence, cross-attention allows decoder tokens (text) to attend to encoder tokens (image patches). Mathematically: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T/\sqrt{d}) \times \mathbf{V}$. Query (Q) from the decoder represents 'what information am I looking for?'; Key (K) and Value (V) from the encoder represent 'what does each patch contain?' The dot product computes similarity scores, softmax normalizes into a probability distribution, and the output is a weighted sum of values. When generating 'dog', attention concentrates on dog-containing patches; for 'beach', attention shifts to background regions.

Autoregressive Text Generation

Caption generation proceeds autoregressively: one token at a time, each conditioned on image features and all previously generated tokens. Starting from a [START] token, the decoder computes probability distribution over the vocabulary (30,522 tokens for BLIP), selects the next token via beam search or sampling, appends it to the sequence, and repeats until generating [END] or reaching maximum length. This sequential dependency enables coherent, grammatically correct outputs but creates a fundamental speed limitation — tokens cannot be generated in parallel.

2.2 Connection to Course Themes

GIGO (Garbage In, Garbage Out)

The GIGO principle takes on special importance in multimodal systems where inputs originate from the physical world. Unlike text data where quality issues (typos, encoding errors) are straightforward to detect, image quality problems are subtle and varied. A low-resolution image, poor lighting, extreme aspect ratio, motion blur, or compression artifacts can significantly degrade caption quality — yet the model produces fluent, grammatically correct output. The failure mode is not an error message but an incorrect or vague caption that may go undetected.

The workshop addresses GIGO through a comprehensive quality analyzer that validates images before captioning. Students implement checks for minimum resolution (ensuring sufficient detail), aspect ratio bounds (detecting cropping issues), brightness range (identifying over/underexposed images), and color variance (catching blank or corrupted images). Beyond input validation, confidence estimation through sampling consistency extends GIGO to outputs — generating multiple captions and measuring agreement as a proxy for model certainty.

Botspeak (Prompt Engineering)

Conditional captioning demonstrates prompt engineering principles for vision-language models. By providing different text prompts ('a photograph of', 'describe the main subject:', 'the mood of this image is'), students observe how prompts guide model behavior, producing systematically different outputs from identical images. This connects to broader Botspeak principles: models are not oracles that divine intent but systems responding to specific patterns. Clear, well-designed prompts are essential. The workshop demonstrates both successful patterns and failure cases (prompts causing repetition loops), building practical intuition.

Computational Skepticism

The evaluation section embodies computational skepticism by emphasizing that outputs should never be blindly trusted. Students implement multiple metrics (BLEU for lexical overlap, semantic similarity for meaning) and analyze conflict cases. A paraphrase ('a dog on the beach' vs 'a canine by the ocean') scores low on BLEU but high on semantic similarity. Conversely, word-order changes ('dog bites man' vs 'man bites dog') score high on BLEU but low on semantic similarity. These exercises teach that evaluation requires human judgment informed by multiple metrics, not blind reliance on any single number.

2.3 Relationship to Real-World Workflows

The workshop deliberately mirrors authentic data science workflows rather than presenting isolated academic exercises:

- **Model Selection:** Students evaluate pre-trained models on multiple criteria: benchmark performance, model size, inference speed, licensing terms, documentation quality, and community support. These are the same considerations practitioners face when choosing models for production.
- **Iterative Experimentation:** The parameter tuning section models systematic hyperparameter variation. Students maintain records of experiments (settings, results, observations), building habits of reproducible research.
- **Multi-Stage Validation:** Quality analyzer and confidence estimation demonstrate that production systems require validation at multiple stages — input validation, output validation, and uncertainty quantification.
- **Error Handling:** The production pipeline section emphasizes robust error handling: catching exceptions, providing informative messages, falling back to safe defaults, and logging for debugging.
- **Documentation:** All code includes comprehensive docstrings and comments, modeling professional documentation standards.

3. Implementation Analysis

3.1 Architecture and Design Decisions

The workshop implementation follows software engineering best practices, balancing educational clarity with production readiness. The architecture separates concerns across components optimized for their purposes:

```
image-captioning-workshop/ ■■■ notebooks/ ■ ■■■ Image_Captioning_Tutorial.ipynb # Main tutorial
(verbose, educational) ■ ■■■ exercises/ ■ ■■■ starter_template.ipynb # Scaffolded practice exercises
■ ■■■ solutions.ipynb # Reference implementations ■■■ app/ ■ ■■■ gradio_demo.py # Interactive web
demonstration ■■■ src/ ■ ■■■ captioner.py # Production-ready reusable module ■■■ docs/ ■ ■■■
pedagogical_report.pdf # This document ■■■ README.md # Project documentation ■■■ requirements.txt #
Pinned dependencies
```

Separation of Educational and Production Code

The tutorial notebook prioritizes educational clarity: extensive comments explain not just what code does but why; intermediate results are printed for inspection; visualizations illustrate concepts. The `captioner.py` module provides identical functionality in production-ready form: clean APIs, proper encapsulation, comprehensive error handling, minimal logging noise. This separation allows students to learn from verbose, explanatory code while seeing how the same logic is structured for deployment.

Key Design Patterns

- **Dataclasses for Structured Results:** `CaptionResult` and `QualityReport` use Python dataclasses for clean, typed return values. Functions return self-documenting objects bundling related information (caption, timing, token count).
- **Union Types for Flexibility:** Functions accept `Union[str, PIL.Image]` inputs, transparently handling URLs, local paths, and PIL Image objects — mirroring real-world flexibility requirements.
- **Optional Detailed Output:** The `detailed=True` parameter switches between simple returns and rich result objects, serving both quick scripting and detailed analysis.
- **Context Managers:** `torch.no_grad()` contexts clearly delineate inference sections, reducing memory usage by ~40%.

3.2 Libraries and Tools Selection

Library	Purpose	Rationale
BLIP	Vision-language model	Best zero-shot performance in size class; permissive license
Hugging Face	Model loading	Industry standard; automatic caching; extensive docs
PyTorch	DL backend	Dynamic graphs for debugging; dominant in research
Gradio	Interactive demo	Rapid prototyping; shareable URLs; no frontend needed
NLTK	BLEU calculation	Reference implementation; well-tested
Sentence-Trans.	Semantic similarity	State-of-the-art embeddings; simple API

3.3 Performance Considerations

Inference Speed

BLIP-base inference requires approximately 100-200ms per image on a modern GPU (NVIDIA T4 or better) and 3-8 seconds on CPU depending on processor speed and image complexity. The workshop is explicitly designed to function acceptably on CPU to ensure accessibility for students without GPU access, though GPU usage is recommended for interactive experimentation where rapid feedback accelerates learning. Batch

processing improves throughput 2-3x through better GPU utilization.

Memory Management

The BLIP-base model requires approximately 1.5GB GPU memory (or system RAM for CPU inference). The workshop explicitly uses `torch.no_grad()` contexts throughout, disabling gradient computation and reducing memory usage by ~40%. For memory-constrained environments, the workshop documents half-precision (FP16) inference via `model.half()` to further reduce requirements.

Caching Strategy

Hugging Face's automatic caching ensures model weights (~1GB) are downloaded only once and loaded from cache on subsequent runs. The workshop explains why first execution takes minutes while subsequent runs start within seconds. Cache location is configurable via `HF_HOME` environment variable for institutional computing environments.

3.4 Edge Cases and Limitations

Known Model Limitations

- **Domain Specificity:** BLIP trained primarily on natural web photographs; performance degrades significantly on medical imaging, satellite imagery, technical diagrams, and text-heavy images.
- **English Only:** Generates English captions exclusively. Multilingual captioning requires different models or post-hoc translation which may introduce errors.
- **Hallucination Risk:** Like all generative models, BLIP can produce plausible but factually incorrect details — describing wrong colors or mentioning absent objects.
- **Repetition Vulnerability:** Certain prompts or parameter configurations trigger repetition loops. Addressed with `repetition_penalty` parameter.
- **Ambiguity Handling:** For ambiguous images, the model commits to one interpretation rather than expressing uncertainty.

Edge Cases Explicitly Handled

- Empty, corrupted, or unreadable image files → graceful error with informative message
- Extremely small images below minimum resolution → quality warning before processing
- Non-RGB images (grayscale, RGBA) → automatic conversion to RGB
- Network failures during URL loading → timeout handling, retry logic, clear error messages
- Missing CUDA/GPU → automatic fallback to CPU with performance warning
- Out-of-memory conditions → caught and reported with resolution suggestions

4. Assessment and Effectiveness

4.1 Validating Student Understanding

The workshop employs multiple complementary assessment mechanisms to validate understanding at different cognitive levels:

Progressive Exercise Sequence

Six exercises of increasing complexity validate mastery from basic application through synthesis:

Exercise	Pts	Level	Success Criteria
1: Basic Caption	10	Applying	Function returns valid caption; handles URL/path/PIL inputs
2: Conditional	15	Applying	Different prompts produce different outputs; can explain why
3: Benchmarking	20	Analyzing	Correct timing; accurate interpretation of tradeoffs
4: Quality Analyzer	25	Creating	All checks implemented; correct thresholds; informative messages
5: Full Pipeline	30	Creating	Complete class with caption(), batch_caption(); error handling
Bonus: Confidence	+10	Evaluating	Multi-sample generation; correct similarity calculation

Self-Assessment Mechanisms

Solutions in a separate notebook enable students to compare implementations against reference solutions after attempting exercises independently. The inherently visual nature of image captioning provides built-in self-assessment — students immediately see whether generated captions make sense for given images. The Gradio demo supports exploratory self-assessment through rapid parameter experimentation without requiring code changes.

4.2 Common Challenges Students May Face

Anticipating common difficulties allows proactive mitigation through instructional design:

- **Environment Setup Friction:** Dependency conflicts, CUDA version mismatches, insufficient disk space for model downloads, and corporate firewall restrictions are common barriers. The workshop provides detailed troubleshooting guidance, recommends Google Colab as a zero-setup alternative, and includes environment verification code.
- **Attention Conceptual Gap:** The cross-attention mechanism is conceptually challenging, especially bridging modalities. The workshop addresses this through multiple explanations: the 'describing a photo' analogy, mathematical formulation, step-by-step examples, and visualization of attention weights.
- **Parameter Interaction Complexity:** Understanding how beam_width, temperature, and top_p interact requires experimentation. The benchmarking exercise provides structured exploration, and the Gradio demo enables rapid parameter sweeps building intuition.
- **Metric Interpretation:** Students initially expect a single 'correctness' metric and may be confused when BLEU and semantic similarity disagree. Deliberate conflict cases teach that evaluation requires informed judgment.
- **Silent Failure Debugging:** Unlike code errors producing stack traces, image quality issues cause subtle degradation without obvious signals. The GIGO section frames proactive input validation as a debugging strategy.

4.3 Addressing Different Learning Styles

The workshop accommodates diverse learning preferences through multiple modalities and pathways:

Learning Style	Workshop Accommodations
----------------	-------------------------

Visual Learners	ASCII architecture diagrams; matplotlib visualizations; Gradio demo with real-time feedback
Reading/Writing	Detailed markdown explanations; comprehensive docstrings; written specifications
Auditory Learners	Designed for instructor narration; Explain-Show-Try supports verbal walkthrough
Kinesthetic	Hands-on code cells; interactive Gradio tuning; build-from-scratch exercises

4.4 Future Improvements and Extensions

Several enhancements are planned to extend the workshop's depth and breadth:

- **Attention Visualization:** Display attention heatmaps showing which image regions influenced each generated word, deepening cross-attention understanding.
- **Fine-tuning Tutorial:** Extend workshop to cover domain adaptation, teaching students to fine-tune BLIP on specialized datasets.
- **VQA Extension:** Expand to Visual Question Answering tasks using BLIP's built-in QA capabilities.
- **Multi-Model Comparison:** Add sections comparing BLIP with GIT, Flamingo, and LLaVA architectures.
- **Automated Grading:** Develop test scripts for automatic submission validation, enabling scalable assessment.
- **Multilingual Extension:** Incorporate multilingual models to address English-only limitation.

4.5 Conclusion

This workshop provides a comprehensive introduction to image captioning with vision-language models, successfully balancing theoretical depth with practical implementation skills. Students progress from understanding fundamental concepts (encoder-decoder architecture, attention mechanisms, autoregressive generation) through hands-on implementation (model loading, caption generation, parameter tuning) to critical evaluation practices (multi-metric assessment, confidence estimation, GIGO validation).

The explicit connections to INFO 7390 course themes reinforce that technical skills must be paired with critical thinking. GIGO principles teach proactive input validation; Botspeak demonstrates prompt engineering for vision-language models; Computational Skepticism builds habits of questioning model outputs. These are not academic abstractions but essential practices for responsible AI deployment.

The workshop's modular design supports multiple learning pathways and teaching contexts. Individual learners can work through the tutorial independently, using the Gradio demo for experimentation and solutions for self-assessment. Instructors can use the materials for live demonstrations, with the Explain-Show-Try structure supporting verbal walkthroughs. The production-ready code in `captioner.py` demonstrates how educational concepts translate to deployment-ready implementations.

Future extensions will expand the workshop's scope while maintaining its educational foundations. Attention visualization will deepen interpretability; fine-tuning tutorials will enable domain adaptation; multi-model comparisons will illustrate the evolving vision-language landscape. These additions will ensure the workshop remains relevant as the field advances.

References

- [1] Li, J., et al. (2022). BLIP: Bootstrapping Language-Image Pre-training. ICML 2022.
- [2] Dosovitskiy, A., et al. (2020). An Image is Worth 16x16 Words. ICLR 2021.
- [3] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.

- [4] Papineni, K., et al. (2002). BLEU: Automatic Evaluation of Machine Translation. ACL.
- [5] Bloom, B.S. (1956). Taxonomy of Educational Objectives. Longmans, Green and Co.